



United International University
Department of CSE

Course Code: CSE 426/CSE 4326

Course Name: Microprocessor, Microcontroller and Interfacing Lab

Experiment no: 01 (A)

Experiment Name: An introduction to Arduino, Interfacing of Gas Sensor using Arduino & Showing the Sensor Data in OLED Display

Submitted by:

Name: Sumaiya Islam Babli

Student ID: 011 221 092

Section: H

Date of Performance: July 25, 2026

Date of Submission: July 26, 2026

Objective:

In this lab, we will learn how to interface a gas sensor with an Arduino and display the sensor data on an OLED display. The experiment is divided into two parts. First, we will learn details about the arduino uno board and how arduino can be used to perform simple hardware projects. In the second part, we will use an important sensor called, gas sensor which detects presence of different gases in the environment. This sensor will be used to detect the presence of gas in the environment, and the Arduino will read the data from the sensor and display it on the OLED display. This lab will teach you how to work with Arduino, sensors and digital displays, as well as how to interface them together to create a complete system. This will definitely give you an idea on how to do mini projects using an Arduino board. Final outcomes of this experiment will be :

- To learn about Arduino boards and the Arduino IDE.
- To understand examples of hardware and software interfacing using an Arduino board.
- To learn how to emulate Arduino projects in Proteus software.
- To interface a gas sensor with an Arduino.
- To use an OLED display to show sensor data.

Apparatus:

- Arduino, USB cable, Ultrasonic sensor (HC-SR04), Breadboard , LDR Sensor, LED , Buzzer , Push Button ,Resistor:10k ohm

Theory:

Logic ICs (AND, NOT):

Logic ICs are digital circuits that perform logical operations. A NOT gate has one input and one output that produces the opposite value of the input (0 becomes 1, and 1 becomes 0). An AND gate has two or more inputs and produces a 1 output only when all inputs are 1, otherwise it outputs 0. These ICs form the basis of digital electronics.

Push Button Switch:

A push button is a simple mechanical switch that allows current to flow when pressed and stops when released. It is used to send signals to microcontrollers or circuits to trigger specific actions like toggling an LED on and off, or activating devices like buzzers.

LED Control:

Light Emitting Diodes (LEDs) emit light when current flows through them. In these experiments, push buttons are interfaced with circuits to control LED on/off states. When the push button is pressed, it completes the circuit and the LED turns on; when released, the LED turns off.

Buzzer:

A buzzer is an audio signaling device that produces sound when electricity passes through it. It is commonly used in alarm systems and notification circuits. By interfacing a push button with a buzzer through an Arduino, we can activate the buzzer sound on demand.

LDR Sensor:

An LDR (Light Dependent Resistor) is a sensor whose resistance changes based on light intensity. When light intensity is high, its resistance decreases, and when light intensity is low, its resistance increases. By interfacing an LDR with Arduino, we can read the analog voltage and make decisions. In this experiment, the LED turns on when light is low (high resistance) and turns off when light is high (low resistance).

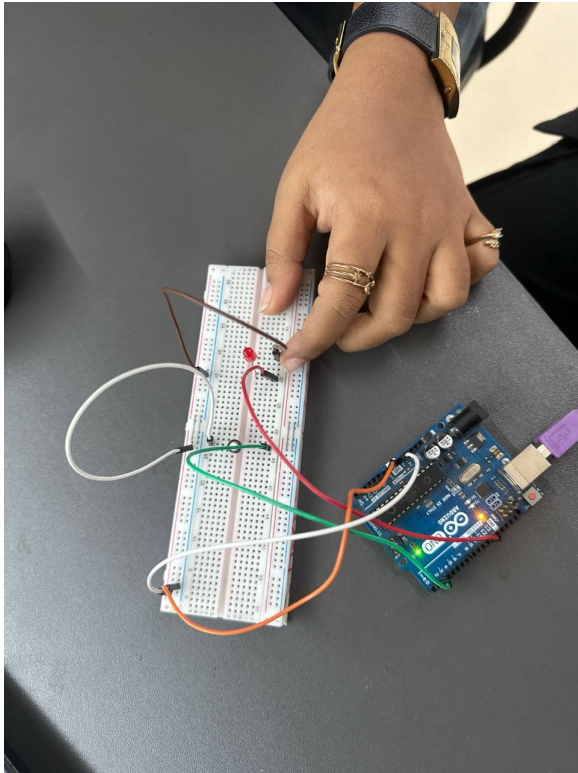
Ultrasonic SONAR Sensor:

An ultrasonic sensor measures distance by sending sound waves and calculating the time taken for echoes to return. It has two main components: a transmitter that sends ultrasonic sound waves and a receiver that detects the reflected waves. The microcontroller calculates the distance using the formula: $\text{Distance} = (\text{Speed of Sound} \times \text{Time}) / 2$. The sensor can measure distances ranging from a few centimeters to several meters and is commonly used in obstacle avoidance systems.

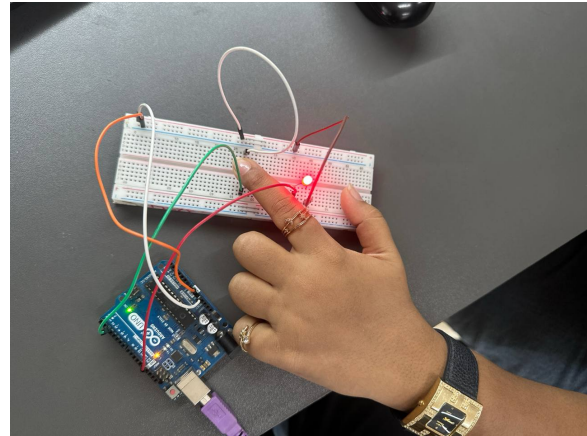
Solution of the mentioned problems:

Task (01) : Use a push button to toggle an LED

In this experiment, I set up a basic interactive circuit using an Arduino Uno, a breadboard, a push button, and an LED. I wired the push button to pin 2 as an input and connected the LED to pin 13 as an output. I wrote a simple program that continuously checks whether the push button is being pressed or released. When the push button is pressed, the Arduino detects the HIGH signal and immediately turns on the LED. When the button is released, the signal changes to LOW, causing the LED to turn off automatically.



(a) LED OFF State



(b) LED ON State

Figure 1: Arduino UNO LED Control Circuit - OFF and ON States

```

1 int buttonState = 0;
2 void setup()
3 {
4     pinMode(2, INPUT);
5     pinMode(13, OUTPUT);
6 }
7
8 void loop()
9 {
10    buttonState = digitalRead(2);
11
12    if (buttonState == HIGH)
13    {
14        digitalWrite(13, HIGH);
15    } else {
16        digitalWrite(13, LOW);
17    }
18    delay(15);
19 }

```

Listing 1: Arduino Button Reader Code

Table 1: Arduino Pin Connections

Arduino Pin	Component Connected	Mode / Function
Pin 2	Push Button	Input (Reads button state)
Pin 13	LED	Output (Controls LED ON/OFF)
GND	Circuit Ground	Common Reference
5V	Push Button Power	Power Supply

Task (02): Use a push button to turn ON a buzzer.

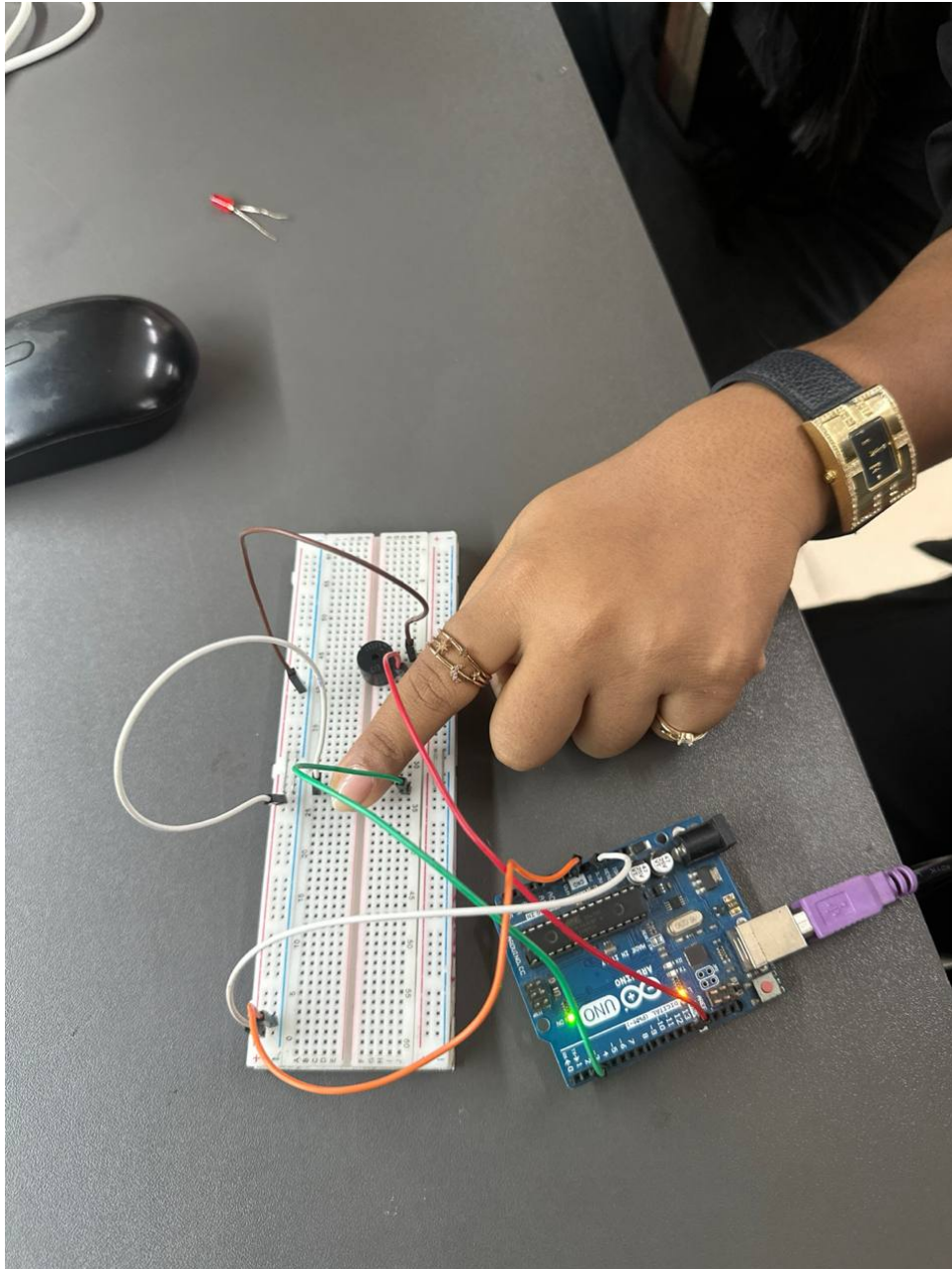


Figure 2: Button Pressed State - Buzzer Activated

```

1 int buttonState = 0;
2
3 void setup()
4 {
5     pinMode(2, INPUT);
6     pinMode(13, OUTPUT);
7 }
8
9 void loop()
10 {
11     buttonState = digitalRead(2);
12
13     if (buttonState == HIGH)
14     {
15         digitalWrite(13, HIGH);
16     } else {
17         digitalWrite(13, LOW);
18     }
19
20     delay(15);
21 }
22
23
24 % Pin Connection

```

Listing 2: Arduino Button Reader Code

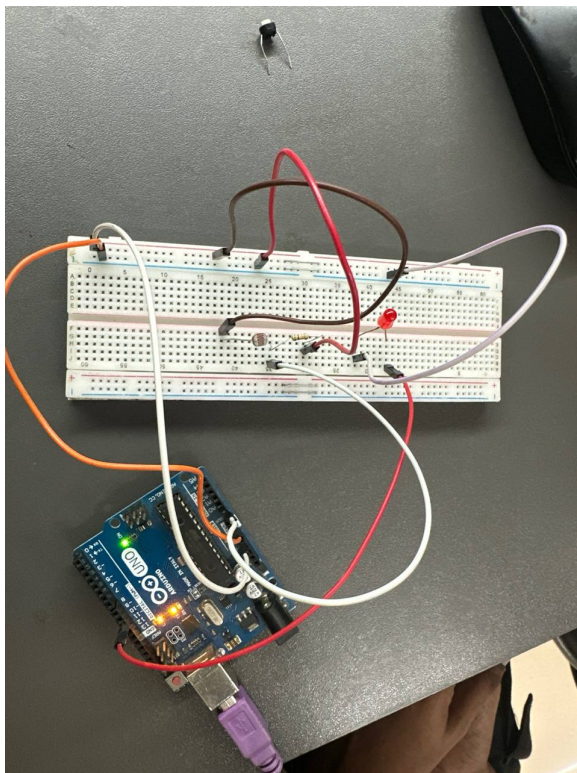
Table 2: Pin Connections for Task 02

Component	Component Pin	Arduino Uno Pin
Push Button	Terminal / Signal	Digital Pin 2 (INPUT)
Buzzer	Positive (+)	Digital Pin 13 (OUTPUT)
Push Button / Buzzer	Ground (GND)	GND
Push Button	VCC / Power	5V

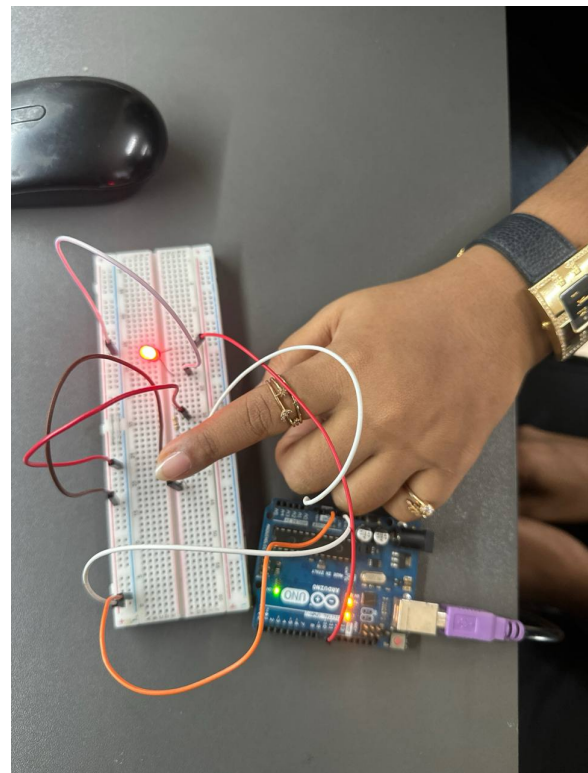
In this experiment, I connected a push button and a buzzer to an Arduino board using a breadboard. I wrote a simple code that constantly checks whether the push button is being pressed or not. When the button is pressed, the code sends a signal to turn the buzzer ON and sound an alarm. As soon as the button is released, the buzzer turns off automatically.

Task (03): Interface an LDR sensor with Arduino in Proteus.

```
1 int sensorValue = 0;
2
3 void setup()
4 {
5     pinMode(A0, INPUT);
6     pinMode(9, OUTPUT);
7     Serial.begin(9600);
8 }
9
10 void loop()
11 {
12     sensorValue = analogRead(A0);
13     Serial.println(sensorValue);
14     analogWrite(9, map(sensorValue, 0, 1023, 255, 0));
15     delay(100);
16 }
```



(a) LDR in Light (LED OFF)



(b) LDR in Dark (LED ON)

Figure 2: LDR Circuit Behavior in Different Lighting Conditions

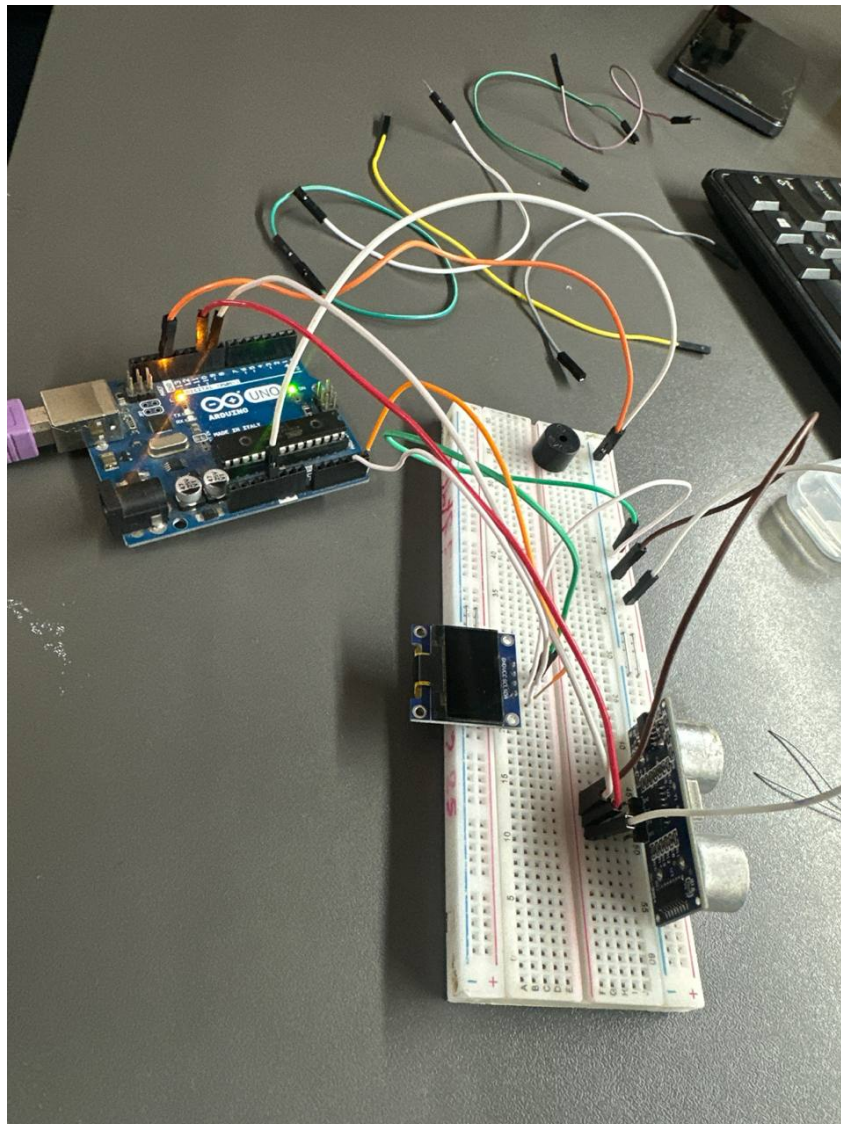
In this experiment, I connected an LDR light sensor and an LED to an Arduino board using a breadboard. The Arduino continuously reads the surrounding light levels from the

LDR through analog pin A0. Using the map() function in the code, the system converts the sensor reading so that decreasing light increases the LED brightness. Finally, pin 9 sends a PWM signal to automatically light up the LED when the sensor is covered or exposed to darkness.

Table 3: Pin Connection Table

Component	Arduino Pin	Description / Mode
LDR Sensor	A0	Analog Input (Reads light intensity)
LED	Pin 9	Digital PWM Output (Controls LED brightness)
Arduino Uno	USB Port	Power Supply & Serial Communication

Task (04): Interface a SONAR sensor with Arduino and measure distance using it. Show the distance in meters.



In this experiment, I aimed to build an automated distance-measuring system by interfacing an ultrasonic (SONAR) sensor and an OLED display with an Arduino board. The goal was to send ultrasonic pulses from the sensor to calculate the distance to a nearby object based on the echo time. I programmed the Arduino to process these sensor readings and wirelessly transmit the calculated distance directly onto the OLED display screen in real-time. Additionally, I set up the breadboard connections to power both the display and sensor, attempting to create a standalone digital measuring device.

Here are the probable reasons why the display likely failed to show any reading in this experiment:

- **Missing OLED Initialization in Code:** The code only calculated distance and printed to the Serial Monitor; it lacked the necessary display libraries (e.g., `Adafruit_SSD1306`) and rendering commands (`display.display()`).
- **I2C Address Mismatch:** The program did not specify or match the required I2C communication address (such as `0x3C` or `0x3D`) needed to talk to the screen.
- **Incorrect I2C Wiring:** The OLED display's SDA and SCL pins were not connected to the designated I2C pins on the Arduino Uno (analog pins A4 and A5).
- **Power & Wiring Issues:** Loose jumper wire connections on the breadboard or insufficient power to drive both the SONAR sensor and display simultaneously prevented proper screen initialization.

```
1 const int trigPin = 9;
2 const int echoPin = 8;
3
4 long duration;
5 float distanceMeters;
6
7 void setup() {
8     pinMode(trigPin, OUTPUT);
9     pinMode(echoPin, INPUT);
10    Serial.begin(9600);
11 }
12
13 void loop() {
14     // Clear the trigPin
15     digitalWrite(trigPin, LOW);
16     delayMicroseconds(2);
17
18     // Trigger the sensor with a HIGH pulse for 10 microseconds
19     digitalWrite(trigPin, HIGH);
```

```

20 delayMicroseconds(10);
21 digitalWrite(trigPin, LOW);
22
23 // Read the echoPin travel time in microseconds
24 duration = pulseIn(echoPin, HIGH);
25
26 // Calculate distance in meters (speed of sound = 343 m/s)
27 distanceMeters = (duration * 0.000343) / 2.0;
28
29 // Print distance to Serial Monitor
30 Serial.print("Distance: ");
31 Serial.print(distanceMeters);
32 Serial.println(" m");
33
34 delay(500);
35 }

```

Table 4: Pin Connection Table for Ultrasonic (SONAR) Sensor

SONAR Sensor Pin	Arduino Pin	Description / Function
VCC	5V	Power Supply (+5V)
Trig (Trigger)	Digital Pin 9	Output (Sends ultrasonic pulse)
Echo	Digital Pin 8	Input (Receives reflected signal)
GND	GND	Ground Connection

How an Ultrasonic Sensor Works?

An ultrasonic sensor (HC-SR04) works by emitting high-frequency sound waves (40 kHz) from its transmitter that travel through the air until they hit an object. When the sound waves strike the object, they bounce back and are received by the sensor's receiver component. The Arduino measures the time duration between sending the pulse and receiving the echo, and calculates the distance using the formula: $\text{Distance} = (\text{Duration} \times \text{Speed of Sound}) / 2$, where the speed of sound is approximately 0.0343 cm/microsecond. This contactless measurement method makes ultrasonic sensors ideal for obstacle detection and distance measurement in robotics and automation applications.

Discussion:

Across these experiments, the main difficulty involved properly wiring hardware components such as push buttons, buzzers, LEDs, LDRs, and ultrasonic sensors onto a breadboard without creating loose connections. Dealing with signal logic and sensor threshold

values in the Arduino code presented an additional challenge, particularly when adjusting code parameters to reliably detect button presses or light changes. Integrating complex modules like the I2C OLED display added another layer of difficulty.